APOSTILA • AULA 19

Otimização de laços e integração do compilador

Projeto MINIC

Foco da aula Reconhecer quando uma transformação de laço preserva o significado do programa e integrar as quatro etapas do compilador em um fluxo testável, documentado e reproduzível.

Referência principal: Aho, Sethi e Ullman, Compiladores: princípios, técnicas e ferramentas, Capítulo 9, especialmente a Seção 9.6; para o fechamento do projeto, Capítulo 1, Seção 1.2, Capítulo 2, Seção 2.8, e Capítulos 8 e 9. Exemplos e contratos de interface são adaptados à especificação acadêmica da linguagem MINIC.

Material de continuidade: as Aulas 11–18 já estabeleceram parsing e AST, semântica, geração de código intermediário, ambiente de execução e fundamentos de otimização. Esta apostila não reabre esses tópicos do zero: usa-os para completar o percurso até um compilador integrado.

Roteiro e objetivos de aprendizagem

Ao final, o estudante deverá conseguir:

- construir a noção de dominância e identificar arestas de retorno e laços naturais em um grafo de fluxo;
- avaliar invariantes, movimentação de código, variáveis de indução e redução de força com atenção a efeitos observáveis e casos-limite;
- explicar por que fusão, desenrolamento e vetorização dependem de provas de dependência e de características do alvo;
- descrever e testar o fluxo integrado MINIC: fonte → tokens → AST → verificação semântica/IR → otimização → código de destino/execução;
- documentar decisões de arquitetura e comparar -O0 e -O1 sem confundir melhoria observada com prova de corretude.

Encontro	Conteúdo central	Conexão com a sequência
21/10	Laços em grafos de fluxo; dominadores; arestas de retorno; invariantes; code motion; indução; redução de força; fusão; desenrolamento; vetorização.	Parte dos blocos básicos e análises de fluxo de dados das Aulas 17–18; prepara a etapa 4 do projeto.
22/10	Integração das quatro etapas; execução; testes; comparação de otimização; documentação e arquitetura.	Fecha o projeto MINIC após lexer, parser/AST, semântica/IR e backend/otimizador; não substitui a aula seguinte sobre interpretação.

1. Onde a Aula 19 se encaixa

O plano organiza o projeto em quatro entregas incrementais: (1) analisador léxico; (2) parser, gramática e AST; (3) tabela de símbolos, análise semântica e IR; (4) gerador de código, otimizador, testes, relatório e manual. A Aula 19 fecha a última etapa e verifica que os contratos entre as anteriores estão funcionando de ponta a ponta.

Na especificação MINIC, o front-end deve produzir uma AST validada; a IR recomendada é código de três endereços (3AC); o otimizador transforma IR preservando o significado; e o gerador produz código de destino executável (por exemplo, assembly ou uma máquina virtual). A implementação pode escolher outro alvo, mas precisa documentá-lo e manter a escolha estável.

```
programa MINIC
    ↓
lexer → tokens (com linha/coluna)
    ↓
parser → AST
    ↓
análise semântica → AST anotada / diagnósticos
    ↓
geração de IR (3AC) → otimizações → IR final
    ↓
gerador de código → assembly / bytecode / VM
    ↓
execução → saída observável
```

Contrato entre fases Cada etapa deve consumir a saída formalmente definida pela anterior e distinguir falha de compilação de falha de execução. Um erro léxico, sintático ou semântico não deve seguir silenciosamente para otimização ou geração de código.

2. Laços vistos como grafos de fluxo

2.1 Blocos, arestas e cabeçalhos

Um grafo de fluxo tem um nó de entrada, nós que representam blocos básicos e arestas que representam transferências possíveis de controle. Um bloco básico é uma sequência máxima de instruções sem desvios internos: a execução entra pelo início e, se prossegue, executa as instruções em ordem até a saída do bloco. Condições, saltos e retornos definem as fronteiras.

Um laço fonte como while (condição) corpo é traduzido em rótulos, teste condicional, corpo e salto de retorno. O ponto de teste é o cabeçalho: cada iteração passa por ele antes de decidir executar o corpo. Isso deixa explícito que zero iterações são possíveis.

```
L_head:
    t0 = i < n
    ifFalse t0 goto L_exit
    ... corpo ...
    i = i + 1
    goto L_head
L_exit:
```

2.2 Dominadores e arestas de retorno

Um nó d domina n quando todo caminho desde a entrada até n passa por d. Todo nó domina a si próprio; o nó de entrada domina todos os nós alcançáveis. Essa propriedade formaliza a ideia de que o cabeçalho de um laço natural é a única porta de entrada para seus nós.

Uma aresta a → h é aresta de retorno (back edge) quando h domina a. O destino h é o cabeçalho e a cauda a é o bloco que retorna. Atenção: uma aresta classificada como “recuo” em uma árvore de busca em profundidade não é automaticamente uma aresta de retorno; a condição de dominância é a definição relevante.

O laço natural associado à aresta a → h contém h e os blocos que podem chegar a a caminhando para trás pelas arestas de predecessores, sem atravessar h como se fosse um novo nó; h domina os nós do laço. Laços com mesmo cabeçalho podem ser unidos. Laços aninhados formam regiões úteis para análises e otimizações.

```
Exemplo abstrato de fluxo:
        ENTRY
          |
          v
      [H: teste] ---- falso ----> [EXIT]
          | verdadeiro
          v
        [B: corpo]
          |
          +------ retorno a H
```

MINIC e redutibilidade Construções estruturadas como while, for, break e continue normalmente geram grafos redutíveis. Saltos arbitrários ou extensões futuras podem criar múltiplas entradas no ciclo; nesse caso, os algoritmos de laços naturais exigem cautela e o compilador pode simplesmente não aplicar certas transformações.

3. Invariantes e movimentação de código

3.1 O que é invariante?

Uma expressão é invariante em relação a um laço quando seu valor não muda entre iterações. Isso exige mais do que “os operandos parecem constantes”: nenhuma atribuição no laço pode alterar operandos, e chamadas ou acessos indiretos não podem invalidar a hipótese. MINIC não especifica ponteiros, mas chamadas a funções ainda podem modificar variáveis globais; uma análise conservadora deve considerar esse efeito se a função puder escrever nelas.

```
for (i = 0; i < n; i = i + 1) {
    dobro = limite * 2;       // candidata invariante, se limite não mudar
    total = total + dobro;
}
```

Se limite não é modificado no laço, dobro = limite * 2 pode ser calculado uma única vez antes do laço. A movimentação reduz cálculos repetidos, mas o compilador só deve movê-lo a um ponto que preserve execução, ordem e efeitos observáveis.

```
dobro = limite * 2;
for (i = 0; i < n; i = i + 1) {
    total = total + dobro;
}
```

3.2 Condições de segurança para code motion

- Os operandos da expressão não mudam no caminho relevante durante as iterações; nenhuma chamada ou escrita indireta invalida essa conclusão.
- A expressão não tem efeitos colaterais (por exemplo, não é uma chamada que imprime ou altera estado).
- A transformação não cria uma avaliação em caminhos em que antes não ocorria. Se o laço executa zero vezes, uma expressão movida para antes do laço passa a ser executada; expressões que podem falhar ou ter efeitos observáveis não podem ser movidas sem prova adicional.
- A ordem observável é preservada: entrada/saída, chamadas, retornos e possíveis erros em tempo de execução não devem ser reordenados indevidamente.
- As regras numéricas do alvo e da linguagem são respeitadas. A especificação MINIC não fecha todos os detalhes de overflow de inteiros; por isso, simplificações algébricas que dependam de ausência de overflow precisam ser justificadas pela semântica adotada, não presumidas.

Exemplo de transformação insegura Mover x = 10 / divisor para antes de um laço que pode executar zero vezes pode introduzir uma divisão por zero que o programa original nunca realizaria. A invariância, isoladamente, não basta: é preciso preservar o domínio de execução da expressão.

3.3 Pré-cabeçalho e múltiplas saídas

Um compilador costuma criar um bloco pré-cabeçalho (preheader): bloco que precede o cabeçalho e é o ponto único de entrada do laço. Cálculos invariantes seguros são movidos para esse bloco. Se houver várias arestas que entram no cabeçalho vindas de fora do laço, um pré-cabeçalho pode reunir essas entradas. Se uma expressão não domina todas as saídas ou poderia deixar de ser executada, a movimentação deve ser restringida.

4. Variáveis de indução e redução de força

4.1 Indução básica e derivada

Uma variável de indução básica muda por um incremento constante em cada iteração, por exemplo i = i + 1. Outra variável j pode ser uma indução derivada se seu valor se relaciona afimmente à básica: j = 4 * i + 3. Reconhecer essa relação permite evitar recomputar multiplicações em cada passo.

```
for (i = 0; i < n; i = i + 1) {
    deslocamento = 4 * i;
    soma = soma + dados[i];
}
```

Uma redução de força transforma, sob condições válidas, a sequência de multiplicações em atualizações aditivas. Pode-se inicializar deslocamento = 0 antes do laço e atualizá-lo com deslocamento = deslocamento + 4 a cada iteração. A indexação ainda precisa ser semanticamente equivalente e respeitar os limites do vetor.

```
deslocamento = 0;
for (i = 0; i < n; i = i + 1) {
    soma = soma + dados[i];
    deslocamento = deslocamento + 4;
}
```

A versão de IR pode usar deslocamento diretamente como índice de bytes apenas se o backend definir esse modelo; na MINIC abstrata, índices de vetor são inteiros em unidades de elementos. Portanto, em uma máquina virtual que indexa elementos, “deslocamento em bytes” não pode ser introduzido sem a conversão correta e a representação declarada do vetor.

4.2 Eliminação de variáveis de indução

Se uma variável derivada só é usada para representar outra variável de indução e não tem uso observável restante, a sua definição pode ser eliminada após substituir os usos pela forma mais simples. A eliminação vem depois da redução de força e da propagação de cópias/constantes; executar as passagens na ordem errada pode deixar código redundante ou apagar uma variável ainda necessária.

Regra prática Primeiro prove a relação entre as variáveis em todos os caminhos do laço; depois transforme; em seguida, rode novamente propagação de cópias e eliminação de código morto. Valide limites, passo negativo, saída antecipada por break e continuação por continue.

5. Transformações de laço: fusão, desenrolamento e vetorização

5.1 Fusão de laços

A fusão combina laços consecutivos com cabeçalhos e limites compatíveis em um único laço. Pode diminuir o custo de controle e melhorar localidade de dados, mas só é segura se a ordem das operações e dependências entre iterações for preservada.

```
// Antes
for (i = 0; i < n; i = i + 1) {
    a[i] = b[i] + 1;
}
for (i = 0; i < n; i = i + 1) {
    c[i] = a[i] * 2;
}

// Fusão candidata
for (i = 0; i < n; i = i + 1) {
    a[i] = b[i] + 1;
    c[i] = a[i] * 2;
}
```

A fusão acima mantém a dependência elemento a elemento dentro de cada iteração, sob a hipótese de que os vetores não se sobrepõem de forma que altere leituras posteriores. Em linguagens com aliasing, ou quando limites/efeitos diferem, a prova precisa ser mais forte. Se o segundo laço depender de toda a execução do primeiro (por exemplo, uma redução global), a fusão pode alterar o resultado.

5.2 Desenrolamento

O desenrolamento duplica o corpo para executar várias iterações por volta do laço. Reduz testes e saltos de controle, mas aumenta tamanho de código e pode piorar cache; é necessário tratar o resto quando o número de iterações não é múltiplo do fator de desenrolamento.

```
// Fator 2: esboço com tratamento do resto omitido
i = 0;
while (i + 1 < n) {
    processa(i);
    processa(i + 1);
    i = i + 2;
}
if (i < n) {
    processa(i);
}
```

Esse esboço é conceitual: a especificação mínima de MINIC não define a função processa nem operações de comparação de ponteiros; o exemplo demonstra controle de iterações. Implementações reais devem garantir que o teste i + 1 não transborde e que break/continue/return dentro do corpo tenham semântica preservada. Um compilador pode escolher não desenrolar laços com controle complexo.

5.3 Introdução à vetorização

Vetorização é a execução de uma mesma operação sobre vários elementos por instrução SIMD (single instruction, multiple data). É uma otimização dependente do alvo: a existência de um laço regular não garante que o compilador possua instruções vetoriais ou que a transformação compense.

Para vetorização, o compilador procura iterações independentes, acessos regulares, tipos suportados e ausência de efeitos/ dependências que exijam a ordem escalar. Operações de redução (como soma acumulada) precisam de uma estratégia específica, pois a associação pode mudar e, com aritmética de ponto flutuante, isso pode alterar arredondamentos. MINIC deve declarar se a vetorização é apenas análise/relatório ou geração SIMD efetiva.

6. Análise do exemplo MINIC

```
int soma(int dados[], int n) {
    int i;
    int total = 0;
    for (i = 0; i < n; i = i + 1) {
        total = total + dados[i];
    }
    return total;
}

int main() {
    int valores[5];
    int resultado;
    valores[0] = 2; valores[1] = 4; valores[2] = 6;
    valores[3] = 8; valores[4] = 10;
    resultado = soma(valores, 5);
    print(resultado);
    return 0;
}
```

O cabeçalho do for verifica i < n; i é a variável de indução básica; total é uma redução dependente da iteração (cada atualização lê o total anterior); dados[i] é um acesso indexado; n e o endereço-base do vetor são candidatos invariantes se não forem alterados. O compilador pode manter o teste e a soma escalar se não provar segurança suficiente para transformações mais agressivas.

Aspecto	Conclusão para o exemplo
Dominância	O bloco do teste domina o corpo do laço e a atualização de i.
Aresta de retorno	A atualização de i volta ao bloco de teste; o cabeçalho domina a cauda da aresta.
Invariante potencial	n e a referência/base de dados, desde que nenhuma escrita/camada de chamada os altere.
Indução	i avança de 1 em 1; total é recorrência/redução, não deve ser tratado como independente.
Limite de otimização	Não presumir que vetor semântica ou aliasing sejam resolvidos sem informação do tipo/alvo.

7. Integração das quatro etapas do projeto MINIC

7.1 Fronteiras e responsabilidades

Etapa	Entrada → saída	Verificações de integração
1. Léxica	Caracteres → tokens com lexema e posição.	Palavras reservadas, comentários, literais, linhas/colunas; erros léxicos impedem fases posteriores.
2. Sintática	Tokens → AST ou diagnóstico sintático.	Precedência/associatividade, blocos, funções, vetores, if/while/for/return; parser consome todos os tokens.
3. Semântica + IR	AST → símbolos, tipos, diagnósticos e IR 3AC.	Escopo, declaração/uso, chamadas, retornos, break/continue, tipos; IR referencia temporários/rótulos válidos.
4. Backend + otimizador	IR → IR otimizada → código de destino → execução.	Semântica preservada, chamada/retorno e controle corretos; diagnósticos de geração e código de saída coerentes.

A especificação propõe interfaces de linha de comando para tokens, AST, símbolos, IR, verificação e alvo, além de códigos de saída distintos para uso inválido, erros léxicos/sintáticos/semânticos, geração e execução. A implementação pode variar, mas deve expor uma interface consistente e documentar desvios. Os nomes exatos dos códigos de saída devem permanecer alinhados ao contrato publicado pelo grupo.

7.2 Roteiro de um programa pelo compilador

1. O lexer reconhece tokens e posições; um literal malformado ou símbolo desconhecido gera erro léxico.
2. O parser consome a sequência e monta nós da AST; token inesperado gera erro sintático com posição e expectativa.
3. A análise semântica resolve identificadores, escopos e tipos; constrói ou anota a AST e valida retorno, argumentos e controle de laços.
4. O gerador de IR traduz expressões e comandos em instruções, temporários, rótulos e saltos; invariantes estruturais da IR são checadas.
5. O otimizador aplica passagens corretas e rastreáveis; modo -O0 desativa otimizações e modo -O1 ativa o conjunto documentado.
6. O backend emite o alvo escolhido; compilador, assembler/linker ou VM transformam esse alvo em execução, conforme arquitetura adotada.
7. A suíte automatizada compara resultado/diagnósticos e status; a documentação registra como reproduzir, testar e executar.

7.3 Decisões de arquitetura que precisam ser explícitas

- Representação intermediária: 3AC em quádruplas, lista de objetos ou outra forma; como representam vetores, chamadas, tipos e rótulos.
- Alvo: assembly nativo, LLVM IR, bytecode ou máquina de pilha própria; ferramentas externas necessárias; plataforma e convenção de chamada.
- Passes do otimizador: quais transformações, em qual ordem, quais pré-condições e limitações; comportamento dos modos -O0/-O1.

- Erros e contratos: códigos de saída, formato de mensagens, se a compilação interrompe na primeira falha ou recupera múltiplos erros.
- Semântica restante: overflow inteiro, divisão por zero, limites de vetores, ordem de avaliação e efeitos de chamadas; decisões não especificadas devem ser declaradas.
- Testabilidade: entradas mínimas, testes por fase, testes de integração, casos válidos e inválidos, regressões e comparação diferencial.

8. Testes de integração e validação de otimizações

8.1 Pirâmide de testes

Nível	Exemplos de casos	O que isola
Unidade por fase	Tokens e posições; AST esperada; diagnóstico semântico; IR de expressão.	Defeito localizado em lexer, parser, semântica ou IR.
Integração curta	Programa válido com função, parâmetro, vetor, if/for, print e return.	Compatibilidade das fronteiras e geração executável.
Integração negativa	Erro léxico, erro sintático, identificador ausente, tipo incompatível, break fora de laço.	Erros não desaparecem nem viram falhas tardias/segfault.
Regressão e diferencial	Executar todos os exemplos com -O0 e -O1.	Otimizações preservam saída e resultado observável.
Alvo/ambiente	Compilar, ligar e executar em ambiente limpo; testar código de retorno.	Dependências, convenção de chamada, arquivos temporários e portabilidade.

8.2 Como demonstrar corretude prática

Para cada teste válido, compare a execução sem otimização e com otimização: mesmo código de saída, mesma saída padrão e mesmos efeitos definidos pelo contrato. Para programas com entrada, use o mesmo conjunto de dados e valide a saída. Para erros, verifique que a fase correta os detecta e que não é emitido executável enganoso. Isso é evidência experimental, não prova formal universal; testes devem ser complementados por argumentos de preservação semântica para cada transformação.

```
minic -O0 programa.mc -o programa_O0
minic -O1 programa.mc -o programa_O1
./programa_O0 < entrada.txt > saida_O0.txt
./programa_O1 < entrada.txt > saida_O1.txt
cmp saida_O0.txt saida_O1.txt
```

A sintaxe acima é ilustrativa: a especificação define opções conceituais como -O0 e -O1, mas cada equipe deve documentar os argumentos realmente implementados. Uma comparação robusta inclui status de saída e stderr quando diagnósticos fazem parte da interface; diferenças de tempo não provam corretude.

8.3 Comparação de desempenho sem conclusões frágeis

- Use vários tamanhos de entrada, inclusive pequenos e grandes; registre compilador, alvo, flags, hardware e condições de execução.
- Faça aquecimento quando aplicável e repita execuções; reporte mediana (ou outra estatística justificada), não apenas a melhor medição.

- Separe tempo de compilação de tempo de execução; mantenha a mesma entrada e o mesmo ambiente para -O0 e -O1.
- Inclua tamanho do código ou memória quando relevante: desenrolamento pode acelerar um caso e aumentar o executável.
- Não declare uma otimização “mais rápida” por inspeção do fonte; apresente medidas reproduzíveis e reconheça variação experimental.

9. Documentação e entrega da etapa final

O relatório e o manual devem permitir que outra pessoa reconstrua e avalie o compilador sem conhecimento tácito do grupo. A documentação de arquitetura deve mostrar a direção dos dados, as interfaces, as representações escolhidas e as limitações conhecidas. Os testes devem ser executáveis e ter resultados esperados mantidos sob controle de versão.

Seção documental	Conteúdo mínimo sugerido
Visão geral	Objetivo, recursos MINIC suportados e fora de escopo, desenho das fases.
Construção e uso	Pré-requisitos, compilação, comandos, opções, exemplos, códigos de saída.
Arquitetura	Formato de tokens/AST/IR, alvo, convenção de chamada e organização dos módulos.
Otimizações	Passes implementados, condições, exemplos antes/depois, limitações e opções.
Testes	Como executar, categorias cobertas, casos de regressão e resultados esperados.
Resultados	Comparação -O0/-O1 com método, entradas, ambiente e métricas.
Limitações	Recursos não implementados, escolhas semânticas e erros conhecidos.

10. Exercícios

Resolva primeiro sem consultar as respostas comentadas. Use MINIC estrito e declare hipóteses quando o contrato não determinar comportamento.

8. No grafo ENTRY → A → B → C, C → B, B → EXIT, identifique o cabeçalho do laço, a aresta de retorno e a condição que confirma que C → B é back edge.
9. Considere o laço: for (i = 0; i < n; i = i + 1) { x = limite * 3; total = total + x; }. Sob quais condições x pode ser calculado antes do laço? Cite duas situações que impedem a transformação simples.
10. No mesmo laço, classifique i e total. Explique por que total não pode ser atualizado por uma soma independente em cada iteração sem preservar a dependência.
11. Fusão: o primeiro laço escreve a[i] = b[i] + 1; o segundo c[i] = a[i] * 2; ambos percorrem 0 ≤ i < n. Que hipótese de dependência/aliasing precisa ser avaliada? Por que isso não basta para vetorização irrestrita?
12. Um laço processa n elementos e é desenrolado por fator 4. Se n = 10, quantos elementos o corpo desenrolado processa e quantos ficam no tratamento residual?
13. Descreva o caminho desde o arquivo-fonte MINIC até a execução, nomeando uma responsabilidade e uma falha detectável em cada fase principal.
14. Proponha um teste diferencial para validar uma otimização de laço. Quais observáveis comparar e quais limitações reconhecer?
15. Explique por que movimentar uma expressão invariante que pode dividir por zero para fora de um laço que executa zero vezes pode alterar o programa.

11. Respostas comentadas

1. Cabeçalho e aresta de retorno

B é o cabeçalho natural. A aresta C → B é uma aresta de retorno se B domina C, isto é, se todo caminho da entrada até C passa por B. No grafo indicado, o único caminho até C é ENTRY → A → B → C, logo B domina C; o ciclo B → C → B constitui o laço natural {B, C}.

2. Movimentação de x

x pode ser movido para antes do laço se limite não for modificado em qualquer caminho relevante, não houver chamada/efeito indireto que o modifique, e a expressão for pura e total sob as regras numéricas adotadas. A transformação simples não é válida se, por exemplo, uma função no corpo pode alterar limite, ou se a expressão puder falhar e o laço talvez execute zero vezes. Deve-se também considerar se x é observável depois do laço quando não há iterações.

3. Indução e recorrência

i é uma variável de indução básica: em cada iteração válida, é incrementada por uma constante. total é uma recorrência de redução, pois cada valor depende do anterior ($total_{i+1} = total_i + dados_i$). Paralelizar ou reordenar essa soma demanda preservar o resultado conforme as regras aritméticas; não se pode simplesmente tratar cada atualização como independente.

4. Fusão e vetorização

É preciso verificar se os vetores podem referir-se à mesma região e se os acessos/ordem mantêm as dependências. Na fusão mostrada, cada c[i] lê a[i] recém-escrito na mesma iteração; fusão conserva essa ordem local se não houver dependência cruzada entre índices/iterações. Vetorização ainda requer demonstrar independência entre lanes, suportar tipos/operações no alvo e considerar efeitos de overflow, limites e possíveis aliases.

5. Desenrolamento por quatro

O corpo principal processa duas iterações completas do grupo de quatro, isto é, 8 elementos. Restam 2 elementos para o laço residual. Os limites devem ser testados sem overflow e todas as saídas de controle do corpo precisam ser preservadas.

6. Fluxo completo

Arquivo → lexer (tokens; detecta símbolo inválido) → parser/AST (estrutura; detecta parêntese ausente) → semântica (símbolos/tipos; detecta nome não declarado) → IR (operações explícitas; valida temporários e rótulos) → otimizador (transformações preservadoras) → backend/alvo (código; detecta falha de emissão) → ligação/VM/execução (resultado; detecta erro de execução). A arquitetura concreta pode agrupar fases, mas deve manter contratos compreensíveis.

7. Teste diferencial

Compile o mesmo programa com otimização desativada e ativada; execute ambos com a mesma entrada; compare saída padrão, status e efeitos definidos pelo contrato, além de confirmar que a compilação não mascarou diagnósticos. Use casos de zero iterações, uma iteração, limites, break/continue e vários tamanhos. Esse teste não cobre todos os programas e não substitui prova/argumento de corretude.

8. Divisão e execução zero

No programa original, a expressão dentro do corpo não é avaliada se a condição inicial for falsa. Movê-la antes do laço faz com que seja avaliada de qualquer modo; um divisor zero pode então causar erro adicional e alterar o comportamento. Preservar a condição de avaliação é parte da equivalência semântica.

12. Síntese para revisão

- Aresta de retorno: cauda → cabeçalho, com cabeçalho dominando a cauda.
- Laço natural: cabeçalho único que domina seus nós e uma aresta de retorno.
- Invariância não basta: code motion deve preservar caminhos de execução, efeitos, erros e regras numéricas.
- Indução básica incrementa por constante; redução de força troca operações caras por atualizações mais simples quando equivalentes.
- Fusão, desenrolamento e vetorização são transformações condicionais, não reescritas mecânicas.
- Integração exige contratos explícitos de fase, testes de ponta a ponta, comparação -O0/-O1 e documentação reproduzível.

Referências e alinhamento curricular

AHO, A. V.; SETHI, R.; ULLMAN, J. D. Compiladores: princípios, técnicas e ferramentas. 2. ed. São Paulo: Pearson/Addison-Wesley. Cap. 1, §1.2; Cap. 2, §2.8; Cap. 8; Cap. 9, com ênfase em §9.6 (loops em grafos de fluxo, dominadores, arestas para trás e laços naturais). Observação de escopo: o plano reúne em 21/10 transformações de laço que se apoiam nos fundamentos de fluxo de dados e otimização já vistos na Aula 18; §9.6 fundamenta diretamente a identificação e estrutura dos laços.

ESPECIFICAÇÃO COMPLETA DA LINGUAGEM MINIC. Versão acadêmica 1.0. Seções 1–2 (escopo e programa), 4–9 (gramática, tipos, escopos, AST e IR), 10–16 (geração, otimização, interface, testes e etapas).